

MODELOS DE CONCORRÊNCIA JAVA VIRTUAL THREADS E GO GOROUTINES EM WORKLOADS I/O-BOUND

Felippe Gustavo de Souza e Silva^{1*}; Prof. Marcos Jardel Henriques²

RESUMO

Este trabalho apresenta uma análise comparativa preliminar de modelos de concorrência voltados a workloads I/O-bound, avaliando o desempenho de Java 25 (com Virtual Threads sob o Project Loom), Go 1.25 (com Goroutines nativas) e Quarkus Native (com OS Threads tradicionais 1:1) sob carga controlada. Os testes locais foram realizados em um processador Apple M4 utilizando o runtime Colima/Linux e a ferramenta k6 para simulação de carga de um Gateway de Pagamentos, integrado com banco de dados PostgreSQL, Redis para cache de idempotência e uma API externa simulando latências de 200 a 500ms. Os resultados demonstram equivalência de throughput e latência média até 200 VUs. No cenário de spike com 500 VUs, Go e Quarkus Native mantiveram o throughput máximo de ~650 req/s, enquanto Java apresentou degradação devido à contenção de lock interna no pool HikariCP. O footprint de memória do Go se manteve o mais eficiente (32 a 78 MB), seguido pelo Quarkus Native (55 a 464 MB) e por fim a JVM Java (789 a 1.935 MB).

Palavras-chave: Virtual Threads; Goroutines; Concorrência; Benchmark; I/O-Bound.

INTRODUÇÃO

O desenvolvimento de sistemas corporativos modernos, em especial gateways de processamento de pagamentos, é caracterizado por cargas de trabalho predominantemente limitadas por operações de entrada e saída (I/O-bound). Tais sistemas dependem de chamadas frequentes a bancos de dados relacionais para escrita de transações, acessos a caches distribuídos para validação de idempotência e requisições HTTP para adquirentes financeiras externas de processamento de cartões de crédito. Historicamente, runtimes tradicionais utilizavam um modelo de concorrência baseado em mapeamento de threads de sistema operacional (1:1), em que cada requisição em trânsito bloqueava fisicamente uma thread de hardware. Sob alta concorrência, esse modelo consome recursos de memória expressivos devido à alocação estática de memória de pilha (stack) para cada thread.

Para contornar o limite de escalabilidade das threads de SO, surgiram modelos de concorrência leve baseados no agendamento cooperativo em espaço de usuário (M:N). O runtime do Go implementou Goroutines desde sua concepção, em que milhares de tarefas leves são mapeadas em um pool enxuto de threads físicas. Em contrapartida, o ecossistema Java recentemente introduziu as Virtual Threads (Project Loom) no Java 21, evoluindo no Java 25 com o JEP 491, o qual mitigou o pinning de carrier threads ao lidar com blocos sincronizados. Surge, contudo, a necessidade de isolar se as disparidades históricas de consumo de memória decorrem do paradigma de threading ou do overhead do próprio runtime da JVM.

O objetivo deste trabalho é comparar empiricamente o desempenho, o footprint de memória e a elasticidade sob carga limite dos modelos de concorrência M:N de Java 25 (Virtual Threads) e Go 1.25 (Goroutines), utilizando o Quarkus Native (OS Threads 1:1) como baseline de contraste para isolar o consumo de recursos da JVM e os impactos do bloqueio direto de threads.

METODOLOGIA

Arquitetura da Solução e Local

Construiu-se um Gateway de Pagamentos modular seguindo os conceitos de Clean Architecture. A lógica de negócio consistiu no recebimento de uma requisição de pagamento, na validação de sua duplicidade em cache, na persistência temporária com status pendente em banco de dados, no envio para processamento em adquirente externa e na atualização final da transação.

O ambiente experimental foi montado em um processador Apple M4 (4 vCPUs, 8 GB RAM) sobre o Colima/Linux em modo nativo ARM64, isolando-se recursos em contêineres Docker. Utilizou-se o PostgreSQL 16 (limite de 300 conexões) montado sobre `emptyDir` para simular a escrita e o Redis 7 para o cache de idempotência. Desenvolveu-se um simulador da API de adquirente em Go, programado para injetar latências uniformes de 200ms a 500ms por request.

Protocolo de Isolamento Científico

Definiu-se um protocolo rigoroso para garantir a isolabilidade estatística e evitar cache hits espúrios de testes sequenciais. Antes de cada rodada de simulação de carga, realizou-se a limpeza completa do cache Redis por meio do comando `redis-cli FLUSHALL`. O gerador de carga (ferramenta k6) foi programado para injetar chaves únicas por meio do header HTTP `X-Idempotency-Key` mapeado como `k6-vu${__VU}-iter${__ITER}`, forçando todas as conexões a percorrerem o fluxo completo de I/O de banco e chamada externa.

Modelos de Concorrência e Runtimes

Configuraram-se três runtimes de teste sob premissas equivalentes de pool de conexões (200 conexões ativas):

1. **Java 25:** Compilado com OpenJDK 25, utilizando Spring Boot 3.5 com suporte nativo a Virtual Threads ativado (`spring.threads.virtual.enabled=true`), pool HikariCP (200 conexões) e Garbage Collector ZGC Generational.
2. **Go 1.25:** Desenvolvido com framework Gin, utilizando Goroutines nativas, driver `pgxpool` (200 conexões) e Garbage Collector padrão do Go.
3. **Quarkus Native:** Desenvolvido com Quarkus 3.15.1, compilado estaticamente em binário nativo via GraalVM Mandrel. Configurou-se pool Agroal (200 conexões), pool de cliente REST (600 conexões) e pool de threads do SO (600 threads).

Executaram-se três cenários de teste controlados por round (três rodadas por cenário): ****Baseline**** (20 VUs, 2 minutos), ****Stress**** (200 VUs, ~2 minutos) e ****Spike**** (500 VUs, 1 minuto). O consumo de memória RAM do processo ativo foi coletado a cada 1 segundo por meio de scripts de estatística de sistema operacional.

RESULTADOS PRELIMINARES

Desempenho e Latência sob Carga Moderada (Baseline & Stress)

Nas rodadas iniciais de 20 VUs e 200 VUs, observou-se uma convergência estatística quase perfeita entre os três modelos de concorrência concorrentes. A latência média manteve-se na faixa de 352ms a 358ms, correspondendo exatamente à média da latência de rede externa do simulador. O throughput das requisições atingiu ~24 req/s no baseline e ~342 req/s no teste de estresse para todos os backends, com taxa de erro em 0,00%, confirmando que até o limite físico de conexões de banco de dados, o paradigma de concorrência não influi no desempenho da entrega.

Tabela 1. Resultados consolidados para o cenário de estresse (200 VUs)

Backend	Latência Média	Mediana p50	Percentil p95	Throughput	RPS	Erros
Java 25 (Virtual Threads)	353,4 ms ± 0,5	353,4 ms ± 0,6	488,4 ms ± 0,3	37.554 req	342,0 req/s	0,00%
Go 1.25 (Goroutines)	351,6 ms ± 0,2	351,8 ms ± 0,1	486,3 ms ± 0,3	37.702 req	343,4 req/s	0,00%
Quarkus Native (OS Threads)	353,5 ms ± 0,6	353,3 ms ± 0,8	488,5 ms ± 0,4	37.545 req	341,8 req/s	0,00%

Nota: Valores representam média ± desvio padrão para N=3 rodadas experimentais.

Comportamento de Elasticidade sob Carga Extrema (Spike)

No cenário de spike (500 VUs concorrentes disputando 200 conexões de banco de dados), observou-se a divergência dos runtimes. Go e Quarkus mantiveram a performance estável de vazão máxima, entregando ~650 RPS com latência média estável de ~353ms (p95 de ~488ms). Em contraste, o Java sofreu severa degradação, limitando-se a 362 RPS com latência média dobrada para 723,8ms (p95 de 917,8ms).

O monitoramento interno da JVM confirmou que a degradação do Java não decorreu de bloqueio de carrier threads (zero eventos registrados com JEP 491). A causa raiz identificada foi a contenção de lock interna do pool de conexões **HikariCP**, que utiliza o bloco sincronizado (**synchronized**) como ponto único de serialização de controle das conexões ativas e em espera. Sob 500 Virtual Threads em concorrência extrema por conexões de banco, essa serialização gerou gargalo cumulativo de enfileiramento. Go (com **pgxpool** usando canais concorrentes lock-free) e Quarkus Native (com pool **Agroal** reestruturado) mantiveram a entrega sem pontos de serialização crítica.

Tabela 2. Resultados consolidados para o cenário de spike (500 VUs)

Backend	Latência Média	Mediana p50	Percentil p95	Throughput	RPS	Erros
Java 25 (Virtual Threads)	723,8 ms $\pm$ 3,1	748,9 ms $\pm$ 3,0	917,8 ms $\pm$ 3,1	21.610 req	362,3 req/s	0,00%
Go 1.25 (Goroutines)	353,6 ms $\pm$ 2,6	353,3 ms $\pm$ 2,6	488,4 ms $\pm$ 1,6	39.176 req	655,3 req/s	0,14%
Quarkus Native (OS Threads)	360,2 ms $\pm$ 5,2	360,0 ms $\pm$ 3,8	495,2 ms $\pm$ 5,0	38.607 req	646,3 req/s	0,50%

Nota: Valores representam média $\pm$ desvio padrão para N=3 rodadas experimentais.

Footprint de Memória RAM (JVM vs Binário Nativo)

As medições de consumo de memória RAM RSS (Resident Set Size) permitiram isolar cientificamente as causas do consumo de hardware:

1. O **Java em modo JVM** consumiu de 789 MB (Baseline) a 1.935 MB (Spike), devido ao overhead de inicialização, áreas de controle como Metaspace (estruturas de classe Spring) e do mecanismo de compilação dinâmica JIT.
2. O **Quarkus Native (AOT)** validou que o consumo de memória do ecossistema Java provém majoritariamente da JVM e não da linguagem, apresentando consumo inicial de apenas 55,5 MB. Sob spike (500 VUs com 600 threads alocadas), o consumo subiu para 463,8 MB devido ao custo de stack fixo das threads do sistema operacional 1:1 (~512KB por stack ativado).
3. O **Go 1.25** demonstrou a maior eficiência de memória, consumindo apenas 32,4 MB em baseline e subindo a no máximo 78,1 MB no pico de carga (500 VUs), tirando proveito da stack dinâmica inicial de ~2KB por Goroutine concorrente.

CONCLUSÃO(ÕES) OU CONSIDERAÇÕES FINAIS

Os resultados preliminares comprovam que Java Virtual Threads e Go Goroutines entregam desempenho equivalente em throughput e latência em cargas moderadas de trabalho I/O-bound. Contudo, em situações de concorrência extrema e disputas por pools finitos, a maturidade de bloqueio concorrente nas bibliotecas de ecossistema (como o HikariCP no Java) torna-se o principal fator limitador, e não o runtime do Loom em si. A compilação nativa em Quarkus comprovou que a linguagem Java é competitiva em memória se isolada da JVM, porém evidenciou o custo do modelo tradicional 1:1 de threads sob alta simultaneidade em comparação com a stack enxuta e dinâmica das Goroutines em Go. Os testes subsequentes em ambiente de nuvem GKE estenderão essas análises sob a ótica econômica de custo operacional real por RPS.

AGRADECIMENTOS

O autor agradece ao Prof. Marcos Jardel Henriques pela orientação acadêmica e pelo apoio técnico fornecido nas revisões dos relatórios de benchmarks locais e nos planos de testes do ambiente em nuvem.

REFERÊNCIAS

GO CORE TEAM. **Go Runtime Scheduler**. Versão 1.25. Disponível em: <<https://go.dev>>. Acesso em: 10 mar. 2026.

ORACLE. **JEP 491: Key Platform Threads in synchronized**. Java Development Kit (JDK) 25. Redwood City: Oracle Corporation, 2026.

RED HAT. **Quarkus - GraalVM Mandrel Native Integration**. Versão 3.15.1. Raleigh: Red Hat Inc., 2025.

SPRING TEAM. **Spring Boot 3.5: Virtual Threads Configuration Guide**. Palo Alto: VMware Tanzu, 2026.

¹ Especializando em Engenharia da Computação. USP/Esalq, Piracicaba - SP. *E-mail autor correspondente: felippe-gustavo@hotmail.com

² Doutor em Engenharia de Software. Orientador USP/Esalq, Piracicaba - SP. E-mail: marcos.henriques@usp.br